İTÜAS OTONOM // YAZILIM DEPARTMANI

Aday Değerlendirme & Teknik Yeterlilik Şartnamesi

Dönem: 2026 Alım Süreci
Kapsam: Algoritma & Matematik

KABUL EDİLEN DİLLER	HARİCİ BAĞIMLILIK	TESLİM FORMATI	SÜRE / SON TESLİM
C, C++ veya Python 3.x	Kesinlikle Yasak (Stdlib)	GitHub Repo veya .zip	14 Kasım / 17:00

Yönerge: Bu sınav adayın algoritma, otonom seyir matematiği ve temiz kodlama prensiplerini ölçer. Her soru için girdi-çıktı formatı, doğrulanmış test verileri ve beklenen sonuçlar verilmiştir. Kodlarınız bu test verileriyle doğrudan çalıştırılabilir olmalıdır. Ödevi tamamen sizin yapmanızı beklemiyoruz yapay zeka tabiki kullanın fakat yaptığınız işin ne olduğunu veya nasıl yapıldığını öğrenin

Soru 1: Haversine Mesafesi ve Baş Açısı (Kerteriz) Hesabı

Navigasyon

Senaryo & Problem: Tekne A koordinatında bulunmakta ve hedef B waypoint'ine gitmelidir. Küresel Dünya yüzeyinde düz çizgi Öklid hesabı ciddi sapmaya yol açar.

Yapılması Gerekenler:

- `calculate_navigation(lat1, lon1, lat2, lon2)` prototipinde bir fonksiyon yazın (R = 6371000 m).
- Haversine formülü ile iki koordinat arası mesafeyi **metre** cinsinden hesaplayın.
- Büyük daire başlangıç kerteriz açısını (Initial Bearing / Azimuth) formüle edin ve sonucu [0°, 360°) aralığına normalize ederek tam kuzeye göre yönü bulun.

> **Örnek Test Verisi & Beklenen Çıktı:** Girdi: Tekne A = (41.1054° N, 29.0238° E) | Hedef B = (41.1082° N, 29.0295° E) Beklenen Mesafe : ≈ 568.4 metre (±1m tolerans) Beklenen Kerteriz: ≈ 56.4° (Kuzey doğu yönelimi)

Soru 2: Açı Sarımı Normalizasyonu ve Dümen Kontrolü (P-Controller)

Kontrol

Senaryo & Problem: Denizde pusula yönü 0°-360° döner. Tekne 355° iken hedef 10° ise tekne +345° dönmemeli; en kısa yönden saat yönünde +15° dönmelidir. Ayrıca dümen açısı fiziksel olarak [-45°, +45°] ile sınırlıdır.

Yapılması Gerekenler:

- `compute_rudder_angle(current_yaw, target_heading, Kp=1.5)` fonksiyonu tanımlayın.
- Açısal hatayı hesaplayıp mutlaka [-180°, +180°] aralığına normalize edin (pozitif: sancak/sağ, negatif: iskele/sol).
- Hata ile K_p katsayısını çarparak hedef dümen açısını bulun; sonucu [-45°, +45°] aralığına kırpın (clamp).

> **Test Vakaları:** Vaka 1: current=350°, target=10° → En Kısa Hata: +20.0° → Rudder (Kp=1.5): +30.0° Vaka 2: current=20°, target=340° → En Kısa Hata: -40.0° → Rudder (Kp=1.5): -45.0° (Doyum) Vaka 3: current=5°, target=5° → En Kısa Hata: 0.0° → Rudder (Kp=1.5): 0.0°

Soru 3: Gürültülü GPS Filtreleme (Moving Average & Outlier Rejection)

Filtreleme

Senaryo & Problem: Çok yollu yansımalar nedeniyle GPS verisinde anlık fizik dışı koordinat sıçramaları olmaktadır. Teknenin azami sürati 12 m/s'dir; 1 saniyede bu mesafeden fazla atlayan veriler bozuk kabul edilmelidir.

Yapılması Gerekenler:

- Bir `GPSFilter` sınıfı yazın; 5 elemanlı bir kayan pencere (Moving Average, N=5) tutsun.
- Yeni gelen koordinatın, önceki geçerli ölçüme göre mesafesini kontrol edin. 1 saniyede > 12m sıçrama varsa veriyi "REJECTED" olarak işaretleyin ve pencereye eklemeyin.
- Geçerli veriyi pencereye ekleyip o anki filtrelenmiş (enlem-boylam ortalaması) konumu döndürün.

> **Test Akışı (Zaman serisi: (t_sn, lat, lon)):** T1: (1, 41.10000, 29.00000) → GEÇERLİ (İlk nokta) T2: (2, 41.10005, 29.00005) → GEÇERLİ (Mesafe ≈ 7.0 m < 12 m) T3: (3, 41.10500, 29.00500) → SIÇRAMA (Mesafe ≈ 690 m > 12 m) → REDDEDİLDİ (Outlier) T4: (4, 41.10010, 29.00010) → GEÇERLİ (T2'ye göre ≈ 7.0 m) → Filtre Çıktısı Alınır

Soru 4: Otonom Görev Durum Makinesi (Finite State Machine)

Durum Makinesi

Senaryo & Problem: Tekne görev süresince rastgele davranamaz; deterministik durum geçiş kurallarına uymalıdır. Tanımlı durumlar: `IDLE`, `NAVIGATING`, `OBSTACLE_AVOIDANCE`, `DOCKING`, `EMERGENCY_STOP`.

Durum Geçiş Kuralları:

- `IDLE` modunda iken `CMD_START` gelirse → `NAVIGATING`
- `NAVIGATING` modunda iken `OBSTACLE_NEAR` gelirse → `OBSTACLE_AVOIDANCE`
- `OBSTACLE_AVOIDANCE` modunda `PATH_CLEAR` gelirse → `NAVIGATING`
- `NAVIGATING` modunda `DESTINATION_REACHED` gelirse → `DOCKING`
- Herhangi bir anda (`DOCKING` hariç) `KILL_SWITCH` veya `CRITICAL_FAIL` gelirse → `EMERGENCY_STOP`

Görev: `MissionFSM` sınıfı yazın; olay listesini sırayla işletip durum geçiş loglarını ekrana yazdırın.

> **Sahte Olay Dizisi & Beklenen Geçişler:** Olaylar: ["CMD_START", "OBSTACLE_NEAR", "PATH_CLEAR", "DESTINATION_REACHED"] Beklenen Çıktı: [STATE CHANGE] IDLE -> NAVIGATING (Sebep: CMD_START) [STATE CHANGE] NAVIGATING -> OBSTACLE_AVOIDANCE (Sebep: OBSTACLE_NEAR) [STATE CHANGE] OBSTACLE_AVOIDANCE -> NAVIGATING (Sebep: PATH_CLEAR) [STATE CHANGE] NAVIGATING -> DOCKING (Sebep: DESTINATION_REACHED)

Soru 5: Izgara Haritada En Kısa Rota (A* veya BFS Algoritması)

Graf Arama

Senaryo & Problem: Bir koy veya gölet 0 ve 1'lerden oluşan 8×8 bir matris olarak verilmiştir (0: su/açık yol, 1: duba/ada engeli). Tekne yatay ve dikey (4-yönlü) hareket edebilir.

Yapılması Gerekenler:

- Başlangıç [0, 0] hücresinden Hedef [7, 7] hücresine giden yolu A* veya BFS ile bulun.
- Ziyaret edilen hücre listesini `[(0,0), (0,1), ..., (7,7)]` ve toplam adım sayısını döndürün. Yol yoksa None döndürülmelidir.

> **Sahte Izgara Harita (8x8 Grid, Python List formatında):** `grid = [ [0, 0, 0, 0, 0, 1, 0, 0], [1, 1, 0, 1, 0, 1, 0, 0], [0, 0, 0, 1, 0, 0, 0, 1], [0, 1, 1, 1, 1, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0, 0], [1, 1, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 1, 1, 1, 0], [0, 0, 0, 0, 1, 0, 0, 0] ] Start: (0, 0) | Goal: (7, 7)` → Algoritmanız engele basmadan optimum yolu üretmelidir.

Soru 6: Poligon Güvenlik Alanı Kontrolü (Point-in-Polygon Geofencing)

Hesaplamalı Geometri

Senaryo & Problem: Teknenin yarışma alanı olan poligonun dışına çıkması diskalifiye sebebidir. Konveks veya konkav olabilen bir poligon için teknenin içeride olup olmadığı anlık denetlenmelidir.

Yapılması Gerekenler:

- `is_inside_geofence(point, polygon_vertices)` fonksiyonunu Ray-Casting (Işın Gönderme) yöntemiyle yazın.
- Noktadan sonsuza yatay bir ışın çekin; poligon kenarlarını tek sayıda kesiyorsa `True` (İçeride), çift sayıda kesiyorsa `False` (Dışarıda) döndürün.

> **Sahte Poligon Köşeleri & Test Noktaları:** `Poligon = [(0, 0), (100, 0), (100, 100), (50, 60), (0, 100)] (Konkav Havuz) Nokta 1: (50, 30) → Beklenen: True (Havuzun içinde) Nokta 2: (50, 80) → Beklenen: False (Konkav girinti nedeniyle sahanın dışında!) Nokta 3: (120, 50) → Beklenen: False (Havuzun tamamen dışında)`

Soru 7: NMEA 0183 GPS Parser ve XOR Checksum Doğrulama

Protokol & Parsing

Senaryo & Problem: GPS alıcısı seri porttan ham ASCII dizgileri iletir. Sinyal bozulmalarını engellemek için NMEA standardındaki XOR Checksum denetlenmeli ve geçerli veriler ayrıştırılmalıdır.

Yapılması Gerekenler:

- `parse_gprmc(sentence)` fonksiyonu yazın. $ ile * arasındaki tüm karakterlerin bitwise XOR (^) toplamını alıp hex formatında sondaki 2 haneli sağlama ile karşılaştırın.
- Eşleşiyorsa: Enlem ve Boylamı Ondalık Dereceye (DD = Deg + Min/60) çevirin; sürati (knot) ve rotayı (track angle) float olarak sözlük/yapı halinde döndürün. Uyuşmuyorsa None döndürün.

> **Ayrıştırılacak Ham Test Verisi:** Cümle 1 (Geçerli): "$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230324,003.1,W*6A" → Checksum '6A' doğrulanmalı. Enlem: 48 + 07.038/60 = 48.1173° N | Hız: 22.4 knot Cümle 2 (Bozuk): "$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230324,003.1,W*FF" → Checksum tutmaz → Fonksiyon hata fırlatmalı veya None döndürmelidir.

Soru 8: Rota Sapma Mesafesi (Cross-Track Error - CTE Hesabı)

Vektör Geometrisi

Senaryo & Problem: Tekne A(x_1, y_1) ile B(x_2, y_2) waypointleri arasındaki doğru rotayı takip etmek zorundadır. Ancak akıntı sebebiyle P(x_p, y_p) noktasına sürüklenmiştir.

Yapılması Gerekenler:

- `compute_cte(p_start, p_end, p_boat)` fonksiyonu yazın.
- AB doğrusu ile tekne pozisyonu P arasındaki dik mesafeyi 2B vektör çapraz çarpımı ile hesaplayın: `d = |(B - A) × (A - P)| / |B - A|`.
- İşaret Kuralı: Tekne rotanın solunda (İskele) ise negatif (-), sağında (Sancak) ise pozitif (+) işaret üretmelidir.

> **Test Verileri & Beklenen Değerler:** Rota: A(0, 0) -> B(100, 0) (X ekseni boyunca doğu yönünde hat) Tekne 1: P(50, 15) → CTE = +15.0 metre (Rotanın 15m sağında/sancak tarafında) Tekne 2: P(30, -20) → CTE = -20.0 metre (Rotanın 20m solunda/iskele tarafında) Tekne 3: P(80, 0) → CTE = 0.0 metre (Tam hat üzerinde)

Soru 9: Güvenlik Gözlemcisi (Watchdog Heartbeat Monitor)

Eşzamanlılık / Emniyet

Senaryo & Problem: Yer istasyonundan her 500 ms'de bir canlılık sinyali (Heartbeat) gelmektedir. Haberleşme koptuğunda tekne başıboş kalmamalı, en fazla 1500 ms sonra motorları durdurmalıdır.

Yapılması Gerekenler:

- `Watchdog` sınıfı tanımlayın. `receive_heartbeat(timestamp_ms)` ve `check_safety(current_time_ms)` metodları oluşturun.
- İki sinyal arası fark > 1500 ms olduğunda `FAILSAFE_TRIGGERED` durumunu aktif edin ve motor gücünü `0.0` yapın.

> **Sahte Zaman Damgalı Mesaj Girişi:** `t=0 ms : receive_heartbeat() → Durum: OK t=500 ms : receive_heartbeat() → Durum: OK t=1000 ms : receive_heartbeat() → Durum: OK (Sinyal koptu...) t=2000 ms : check_safety() → Geçen süre: 1000 ms (<=1500) → Durum: OK t=2600 ms : check_safety() → Geçen süre: 1600 ms (> 1500) → ALARM: FAILSAFE_TRIGGERED!`

Soru 10: İki Şamandıra Arasından Geçiş Kapısı (Gate Heading Vector)

Vektör Geometrisi

Senaryo & Problem: Parkurda sol tarafta kırmızı şamandıra K(x_1, y_1) ve sağ tarafta yeşil şamandıra Y(x_2, y_2) bulunmaktadır. Tekne kapının ortasından ve kapı çizgisine tam dik geçmelidir.

Yapılması Gerekenler:

- Kapının orta noktasını hesaplayın: `M = ((x1+x2)/2, (y1+y2)/2)`.
- K'den Y'ye olan kapı vektörüne dik olan normal vektörü bulun: `v_gate = (x2 - x1, y2 - y1)` → `normal = (-(y2 - y1), x2 - x1)`. Normal vektörü normalize edip yaklaşma açısını (heading) derece cinsinden bulun.

> **Test Verisi & Beklenen Çıktı:** `Kırmızı Şamandıra K = (10, 30) | Yeşil Şamandıra Y = (10, 10) Kapı Orta Noktası M : (10.0, 20.0) Kapı Vektörü : (0, -20) → İleri Geçiş Dik Vektörü: (+20, 0) Beklenen Yaklaşma Açısı: Tam Doğu yönü (+X ekseni boyunca, 90.0° veya 0 radyan)`

Değerlendirme Kriterleri & Teslim Yönergesi

Genel Standart

Değerlendirme Kriterleri:

- Algoritmik ve Matematiksel Doğruluk
- Temiz Kod, İsimlendirme ve Yorum Satırları
- Köşe Durumların (Edge Cases) Ele Alınışı
- Çalıştırılabilir Test Scriptleri ve Uyum

Teslim Kuralları:

- Tüm çözümler tek bir GitHub reposunda veya klasörlenmiş `.zip` olarak iletilmelidir.
- Her soru bağımsız çalıştırılabilir olmalıdır (örn. `python soru1.py` veya `g++ soru1.cpp`).
- Sorularda verilen örnek test verileriyle kodların çıktısı doğrulanmalıdır.

BÖLÜM 2: BİLGİSAYARLI GÖRÜ (OPENCV)

Görüntü İşleme, Renk Uzayları, Filtreleme ve Nesne Tespiti

Soru 11: Çoklu Renkte Duba Tespiti ve Sınırlayıcı Kutu (Bounding Box)

OpenCV & Algı

Senaryo & Problem: Su yüzeyinde konumlanmış yeşil, siyah ve kırmızı renkli dikey silindirik işaret dubaları bulunmaktadır. Bu dubaların görüntü işleme algoritmaları ile tespit edilip kutulanması gerekmektedir.

Yapılması Gerekenler:

- Girdi görselini indirmek için test verisini kullanın: Google Drive Görsel Bağlantısı
- Görüntüyü HSV renk uzayına dönüştürerek Yeşil, Siyah ve Kırmızı için uygun maskeler oluşturun.
- `cv2.findContours` ve `cv2.boundingRect` ile 3 dubanın çevresine kutu çizdirip rengini üzerine yazdırın.
- Çıktıyı `output_buoys.png` olarak kaydedin ve piksel koordinatlarını konsola bastırın.

Beklenen Konsol Çıktı Formatı:

```
[DETECTION] Yesil Duba : Bounding Box (x=85, y=360, w=15, h=165) [DETECTION] Siyah Duba : Bounding Box (x=320, y=365, w=16, h=145) [DETECTION] Kirmizi Duba: Bounding Box (x=455, y=362, w=14, h=155)
```

Soru 12: Ufuk Çizgisi Tespiti ve Su Segmentasyonu (ROI Extraction)

Kenar & Çizgi Tespiti

Senaryo & Problem: Deniz üzerindeki hava araçları, binalar veya bulutların sahte engel olarak algılanmasını önlemek için ufuk çizgisinin bulunup sadece su yüzeyinin (ROI) ayrıştırılması gerekir.

Yapılması Gerekenler:

- Sayfa 7'deki deniz/gün batımı görselini (https://png.pngtree.com/background/20230716/original/pngtree-sunset-over-the-sea-skyline-seashore-greece-photo-picture-image_4243206.jpg) girdi olarak kullanın.
- Gauss bulanıklaştırma, `cv2.Canny` ve `cv2.HoughLinesP` ile deniz ile gökyüzünü ayıran ufuk doğrusunu tespit edin.
- Ufuk çizgisinin üstünü tamamen siyah (0) yapan, altını ise orijinal bırakan ikili bir maske üretin. Çıktıyı `water_roi.png` olarak kaydedin.

Beklenen Çıktı:

```
[HORIZON] Tespit Edilen Dogru Denklemi: y = m*x + n (Yatay egim |m| < 0.1) [ROI] Maskeleme basarili. Su bolgesi alani basariyla ayrildi.
```

Soru 13: Güneş Parıltısı ve Parlak Su Yansımalarını Bastırma (Glare Filter)

Morfoloji & Ön İşleme

Senaryo & Problem: Güneş ışığının doğrudan su yüzeyine vurması kamerada parlaklık doygunluğuna (Value ≈ 255) yol açar ve nesne tespiti algoritmalarının konturlarını parçalar.

Yapılması Gerekenler:

- Güneşli deniz görselini (`https://png.pngtree.com/background/20230716/original/pngtree-sunset-over-the-sea-skyline-seashore-greece-photo-picture-image_4243206.jpg`) girdi olarak alın.
- HSV veya LAB renk uzayında eşikleme yaparak parıltılı (yüksek parlaklık, düşük doygunluk: V > 220, S < 40) piksel maskesini izole edin.
- Morfolojik operatörler (closing/dilation) ve medyan filtre (veya `cv2.inpaint`) uygulayarak bu beyaz patlamaları gideren bir ön işleme filtresi yazın. Temizlenmiş resmi `glare_reduced.png` olarak kaydedin.

> **Beklenen Çıktı Metrikleri:** `[GLARE] Filtrelenen Pariltili Piksel Sayisi: Hesaplanmali (> 5000 piksel) [GLARE] Islem tamamlandi, parildayan dalga pikselleri yumusatildi.`

Soru 14: Duba Taban Hattı Tespiti ve Su Yansımasını Ayırma

Gradyan & Segmentasyon

Senaryo & Problem: Aşağıdaki gerçek dünya karesinde görüldüğü gibi, sarı şamandıranın su yüzeyiyle temas ettiği taban hattında suyun rengi, köpük ve yansıma nedeniyle kontur karmaşası oluşur. Otonom tekne mesafesini duba tabanına göre hesapladığı için taban seviyesi hatasız bulunmalıdır.

Yapılması Gerekenler:

- Yandaki gerçek şamandıra görselini(https://pmsdockmarine.com/wp-content/uploads/2023/06/buoy2-850x478.jpg) girdi alın.
- Sarı renk maskesi ile şamandıranın gövdesini kabaca izole edin.
- Dikey Sobel gradyanı (`cv2.Sobel`, y-yönü) veya renk doygunluk farkı analizi yaparak şamandıranın deniz suyu ile kesiştiği alt temas hattını (Waterline Y-koordinatı) tespit edin.
- Su altında kalan bulanık gölge/yansıma kısmını kırparak sadece su üstündeki şamandıraya oturan doğru sınır kutusunu (Bounding Box) yeşil renkle çizin.

> **Beklenen Konsol Çıktısı:** `[WATERLINE] Tespit Edilen Duba-Su Temas Hatti Seviyesi: y ≈ 780-820 piksel [WATERLINE] Kirpilmis Duzgun Sinir Kutusu: [x, y, w, h] konsola yazdirilmalidir.`